Coursework Details

SET09102 Coursework: Peer-to-Peer Rental Marketplace

⚠ Important Assessment Information:

- Report: 20% of module grade - marked quantitatively (marks for included items)
- Presentation: 50% of module grade - marked qualitatively (ability to explain work)
- Duration: 4-6 weeks (half semester project)
- Type: Individual project (not group work)

1. Assignment Overview

In this coursework, you will build a **"Library of Things"** peer-to-peer rental marketplace using .NET MAUI. This is not just about building an app that works - it's about demonstrating professional software engineering practices including clean code, proper architecture, comprehensive testing, and automated deployment.

> **Project Context:** Community members can list items (tools, camping gear, board games) for rent, discover items near their location using spatial search, request rentals from item owners, and provide feedback through reviews.

Real-World Example: Sarah has a power drill she rarely uses. Instead of it sitting idle, she lists it on the marketplace. Mike needs to hang shelves but doesn't want to buy a drill. He searches "drill near me," finds Sarah's listing 2km away, requests a rental for the weekend, and picks it up after Sarah approves. After returning it, Mike leaves a 5-star review.

Key Engineering Challenges

- **Multi-State Rental Workflow:** Handle complex state transitions (Requested → Approved → Rejected, Out for Rent → Overdue → Returned → Completed)
- **PostGIS Spatial Querying:** Location-based discovery ("Find items within 5km")
- **Repository Pattern:** Abstract data access and spatial logic
- **Service Layer:** Business logic separate from ViewModels
- **State Pattern** (Bonus): Manage rental state machine
- **MediatR Event Handling** (Bonus): Decoupled workflow orchestration

2. Learning Objectives & Module Alignment

This coursework addresses all four learning outcomes for SET09102:

Learning Outcome	How Demonstrated in Project	Assessment Method
LO1: Design and create high-quality, maintainable code	Implement Clean Code principles (SRP, DRY, Open/Closed), SOLID principles, design patterns (MVVM, Repository, Service Layer)	**Report:** Code submission **Presentation:** Explain architectural decisions
LO2: Implement software development processes to optimise quality	Set up CI/CD pipeline with automated testing, use Git workflow with meaningful commits, apply testing strategies	**Report:** CI/CD artifacts, test coverage reports **Presentation:** Explain development workflow
LO3: Evaluate and apply software quality assurance metrics	Implement comprehensive unit tests with xUnit, achieve measurable test coverage (>60%), use code quality tools (SonarCloud optional)	**Report:** Test suite and coverage metrics **Presentation:** Explain testing strategy and quality metrics
LO4: Demonstrate competent and ethical use of AI tools	Document AI tool usage (GitHub Copilot, ChatGPT, etc.), show how AI output was validated and controlled, demonstrate understanding of AI-generated code	**Report:** AI usage documentation **Presentation:** Explain and defend AI-assisted code

Expected Outcomes

The list below shows what you are aiming for. Your grade will depend on how completely and how ell you achieve these things.

- Functional .NET MAUI application extending StarterApp
- Clean architectural patterns (MVVM, Repository, Service Layer)
- Comprehensive test suite with proper fixtures
- Working CI/CD pipeline with GitHub Actions
- Professional documentation
- Transparent and responsible AI tool usage

3. Technical Stack

Required Technologies

- **.NET 10.0:** Latest framework version
- **.NET MAUI:** Cross-platform mobile UI framework
- **PostgreSQL 16 with PostGIS:** Spatial database (via Docker/API)
- **Entity Framework Core:** ORM and migrations
- **xUnit:** Testing framework
- **GitHub Actions:** CI/CD pipeline
- **Docker Compose:** Development environment

Development Environment

- Visual Studio Code with C# Dev Kit
- Docker Desktop
- Android Emulator or Windows target
- StarterApp (Docker-based setup)

Architecture Patterns

- MVVM (Model-View-ViewModel)
- Repository Pattern
- Service Layer
- (Bonus) State Pattern
- (Bonus) MediatR/CQRS Lite

4. Getting Started

Step 1: Download and unzip StarterApp

1. Click Download StarterApp ZIP
2. Extract the ZIP file to your desired location (e.g., `~/Projects/StarterApp` on Mac or Linux; `C:\Projects\StarterApp` on Windows)

The StarterApp provides:

- Docker Compose setup with MAUI app container + PostgreSQL container
- Basic authentication and navigation (already implemented - use this for API auth)
- Entity Framework Core with migrations
- Connection to containerised PostgreSQL 16

> **Apple Silicon Macs:**
> Edit the `docker-compose.yml` file and uncomment the line `platform: linux/amd64` as we did in the setup tutorial.
>
> Also edit the file `StarterApp.csproj` and uncomment the line `<RuntimeIdentifier>android-arm64</RuntimeIdentifier>`. This line tells the compiler to use the ARM64 version of the .NET runtime which is correct for the version of the emulator you are using.

Step 2: Configure Database Connection

StarterApp provides an `appsettings.json.template` in the StarterApp.Database for you to configure. For local development, the app connects to PostgreSQL using:

```
Host=10.0.2.2:5432;Username=app_user;Password=app_password;Database=appdb
```

`10.0.2.2` is the IP address of the Docker host machine.

Once you have updated the connection string, change the name of the file to `appsettings.json` (i.e. remove the `.template`).

Check that the database is accessible using the database option in the left-hand menu in VS Code. You should see a Postgres connection named `AppDB`

Step 3: Reset the container environment

In StarterApp, Docker Compose is configured to create the required tables in the database when you first build the container. However, there may be conflicts with the previous exercise. To prevent this, we will tell Docker Compose to remove the existing database and recreate it. In a terminal window on the host computer, change into the project home directory and execute the following command:

```
docker compose down -v
```

This will stop and remove all old containers, networks, and volumes and set up new ones for StarterApp. You should only have to do this once if you are working on your own computer. If you are using one of the lab machines, you can skip this step if this is a fresh session and you are not coming directly from the setup tutorial

Step 4: Open in VS Code Dev Container

1. Open the project folder in VS Code
2. Click "Reopen in Container" when prompted

Step 5: Rebuild everything

> **Note:** The first time they are run, some of these steps take several minutes - please be patient. Wait until each task has finished before starting the next.

1. Activate the command palette with SHIFT+CTRL+P or SHIFT+CMD+P and choose `Dev Containers: Rebuild Container Without Cache`. This task is finished when you can see `Projects: 3` in the status bar. If it looks as though there is a pending task labelled `Android: Analyzing Environment...`, you can ignore it.
2. Use the command palette to start a new terminal window. Then execute the following commands:

   ```
   dotnet clean
   dotnet build -c Debug
   ```

> ⚠ Monitor the build
> **Sometimes the build step can get stuck at the `_CompileResources` step. If this step takes longer that 10s, open another terminal and execute the following steps:**
>
> 1. **Find the process number of the build command with `pgrep -a dotnet`. The process number you need is the one at the start of the line that contains `dotnet build`**
> 2. **Kill the process with the command `kill -9 XXX` where `XXX` is the process number from the previous step**
> 3. **Close the new terminal with the command `exit`**
> 4. **Back in the original terminal, run the same command again.**

Step 6: Test the Starter App

Locate the file `com.companyname.starterapp-Signed.apk` in the `bin/Debug/net10.0-android` folder.

Check that the emulator and the ADB server are running on the host computer. Then execute the following commands:

```
adb uninstall com.companyname.starterapp
adb install -r PATH_TO_YOUR_APK_FILE
```

Once installed, launch the app from the emulator.

Step 7: Initialise Your Repository

Initialise `git` in the project root directory.

Optionally, you can change the name of the app to `RentalApp` so that you get the structure shown below:

```
RentalApp/
├── RentalApp/              # Main MAUI project
├── RentalApp.Database/     # Shared database library
├── RentalApp.Migrations/   # EF Core migrations
└── RentalApp.Test/         # xUnit tests
```

5. Feature Requirements (Tiered)

Features are organised into three tiers that align with grading levels:

Tier 1: Core Features (Required for Pass)

Grade Level: Pass (40-59%)

> **Note:**
>
> StarterApp already provides user authentication (login, registration, user profile). See below for an example of how to switch between local authentication and authentication via the API. You can use similar patterns for all the other features.

1. **User Authentication Integration**
   - Integrate StarterApp authentication with backend API
   - Obtain and store JWT token from API (POST /auth/token)
   - Include token in all authenticated API requests
   - Handle token expiration and refresh
2. **Item Management**
   - Create item listing with title, description, daily rate, category, location
   - View list of all items
   - View detailed item information
   - Update item details (owner only)
3. **Basic Rental Request**
   - Submit rental request for an item
   - View list of rental requests (incoming/outgoing)
4. **MVVM Architecture**
   - ViewModels for all pages
   - ObservableObject for property notification
   - Commands instead of event handlers
   - Clear separation: View → ViewModel → Model
5. **Repository Pattern**
   - IRepository<T> interface
   - Repositories for Items, Rentals, Reviews
   - Abstract data access from ViewModels

Tier 2: Intermediate Features (Required for Merit)

Grade Level: Merit (60-69%)

6. **Location-Based Discovery (PostGIS)**
   - "Find Near Me" feature using GPS coordinates
   - Query items within configurable radius (e.g., 5km)
   - Use PostGIS spatial functions: `ST_DWithin`, `ST_MakePoint`
   - Store locations as `GEOGRAPHY(POINT, 4326)`
   - Abstract spatial logic in LocationService (not in ViewModel)
7. **Rental Workflow Management**
   - Approve/Reject rental requests (owner side)
   - Track rental status transitions:
     - Requested → Approved/Rejected
     - Approved → Out for Rent (on start date)
     - Out for Rent → Returned → Completed
   - Prevent double-booking (validate date overlaps)
   - Calculate total price based on daily rate and duration
8. **Reviews & Feedback**
   - Submit rating (1-5 stars) and comment after rental completion
   - View all reviews for an item
   - Display average rating on user profile
9. **Service Layer**
   - Business logic separate from ViewModels
   - Services: LocationService, RentalService, ReviewService
   - Validate business rules (no double-booking, valid state transitions)
10. **Comprehensive Testing**
    - Unit tests for ViewModels, Services, Repositories
    - Test coverage >60%
    - DatabaseFixture for integration tests
    - AAA (Arrange-Act-Assert) pattern
    - Mock ILocationService for testing

Tier 3: Advanced Features (Bonus - Enhances to Distinction)

Grade Level: Distinction (70-100%)

11. **State Pattern Implementation**
    - Rental state machine
    - States: RequestedState, ApprovedState, OutForRentState, OverdueState, ReturnedState, CompletedState
    - Each state encapsulates valid transitions
    - Prevent invalid state transitions
12. **MediatR Event-Driven Architecture**
    - Commands: RequestRentalCommand, ApproveRentalCommand, ReturnRentalCommand
    - Events: RentalRequestedEvent, RentalApprovedEvent, RentalReturnedEvent
    - Handlers: UpdateDatabaseHandler, NotifyOwnerHandler, AuditLogHandler
    - Demonstrate Open/Closed Principle: Add new handler without modifying existing code
    - CQRS Lite structure with Features folder
13. **Overdue Detection**
    - Automatically detect when rentals become overdue
    - Transition Out for Rent → Overdue state
    - (Optional) Send notifications to borrower
14. **Advanced Spatial Features**
    - Map integration showing item locations
    - Adjustable search radius slider
    - Distance calculation and display
15. **SonarCloud Quality Analysis**
    - Integrate SonarCloud with CI/CD pipeline
    - Pass quality gates
    - Address code smells and security issues

6. Architecture & Design Patterns

Required Folder Structure

```
RentalApp/
├── RentalApp/                      # Main MAUI project
│   ├── Views/
│   │   ├── ItemsListPage.xaml      # Browse items
│   │   ├── ItemDetailPage.xaml     # Item details
│   │   ├── CreateItemPage.xaml     # List new item
│   │   ├── NearbyItemsPage.xaml    # Location search
│   │   ├── RentalsPage.xaml        # Manage rentals
│   │   └── ReviewsPage.xaml        # Submit/view reviews
│   ├── ViewModels/
│   │   ├── ItemsListViewModel.cs
│   │   ├── ItemDetailViewModel.cs
│   │   ├── CreateItemViewModel.cs
│   │   ├── NearbyItemsViewModel.cs
│   │   ├── RentalsViewModel.cs
│   │   └── ReviewsViewModel.cs
│   ├── Services/
│   │   ├── ILocationService.cs     # GPS abstraction
│   │   ├── LocationService.cs
│   │   ├── IRentalService.cs       # Business logic
│   │   ├── RentalService.cs
│   │   ├── IApiService.cs          # API client
│   │   └── ApiService.cs
│   ├── MauiProgram.cs
│   └── RentalApp.csproj
├── RentalApp.Database/             # Shared database library
│   ├── Data/
│   │   ├── AppDbContext.cs
│   │   └── Repositories/
│   │       ├── IItemRepository.cs
│   │       ├── ItemRepository.cs
│   │       ├── IRentalRepository.cs
│   │       ├── RentalRepository.cs
│   │       └── ...
│   ├── Models/
│   │   ├── Item.cs
│   │   ├── Rental.cs
│   │   ├── Review.cs
│   │   ├── User.cs
│   │   └── Category.cs
│   ├── States/                     # (Bonus) State Pattern
│   │   ├── IRentalState.cs
│   │   ├── RequestedState.cs
│   │   ├── ApprovedState.cs
│   │   ├── OutForRentState.cs
│   │   └── ReturnedState.cs
│   ├── Features/                   # (Bonus) MediatR CQRS
│   │   └── Rentals/
│   │       ├── Commands/
│   │       │   ├── RequestRentalCommand.cs
│   │       │   └── RequestRentalHandler.cs
│   │       └── Events/
│   │           ├── RentalCreatedEvent.cs
│   │           └── NotifyOwnerHandler.cs
│   └── appsettings.json
├── RentalApp.Migrations/           # EF Core migrations
│   └── Migrations/
└── RentalApp.Test/                 # xUnit tests
    ├── ViewModels/
    │   └── ItemsListViewModelTests.cs
    ├── Services/
    │   └── RentalServiceTests.cs
    ├── Repositories/
    │   └── ItemRepositoryTests.cs
    └── Fixtures/
        └── DatabaseFixture.cs
```

API Integration

You have two ways to authenticate users in this app. The method that uses the local database is already implemented as a service which implements the `IAuthenticationService` interface. The recommended approach is to create a second service that implements the same interface but which communicates with the API. Either service can be injected as a dependency and the ViewModel does not need to know anything about the implementation. We can use a toggle in the `MauiProgram.cs` file to switch between the two implementations. The code examples below provide you with an implementqation of the API service and the modified `MauiProgram.cs` file. Note that the existing service has been renamed to differentiate it more clearly from the new one.

▶ Click to expand: API Service Implementation
▶ Click to expand: Modification for MauiProgram.cs

MVVM Pattern

Following the MVVM tutorial:

- **Models:** Database entities (Item, Rental, Review)
- **Views:** XAML pages with minimal code-behind
- **ViewModels:** ObservableObject, Commands, business logic orchestration
- **Data Binding:** Connect Views to ViewModels via XAML binding

▶ Click to expand: Example ViewModel

Repository Pattern

Abstract data access:

▶ Click to expand: Repository Pattern Example

Service Layer

Business logic separate from ViewModels:

▶ Click to expand: Service Layer Example

Clean Code: "Smelly" vs "Clean"

Understanding the difference between poor and good code is essential:

> ❌ **Smelly Code (God Object, Mixed Concerns):**

▶ Click to expand: Smelly Code Example

> ✅ **Clean Code (Single Responsibility, Service Layer):**

▶ Click to expand: Clean Code Example

Bonus: State Pattern

For Distinction-level work, implement State Pattern for rental workflow:

▶ Click to expand: State Pattern Example

Bonus: MediatR Pattern

For Distinction-level work, implement MediatR for event-driven architecture:

▶ Click to expand: MediatR Pattern Example

7. Testing Requirements

Test Coverage Expectations

- **Pass (40-59%):** Some unit tests present, >40% coverage
- **Merit (60-69%):** Good test coverage, >60% coverage
- **Distinction (70-100%):** Comprehensive tests, >80% coverage

Testing Framework: xUnit

Following the unit_test tutorial:

- Use xUnit for all tests
- AAA (Arrange-Act-Assert) pattern
- DatabaseFixture for integration tests
- Mock external dependencies (API, GPS)

DatabaseFixture Pattern

▶ Click to expand: DatabaseFixture Example

Example Unit Test

▶ Click to expand: Unit Test Example

Mocking External Dependencies

Mock ILocationService for testing:

▶ Click to expand: Mocking Example

Test Organisation

- One test file per class being tested
- Group related tests using nested classes
- Use descriptive test names: `MethodName_Scenario_ExpectedResult`
- Use `[Theory]` and `[InlineData]` for parameterised tests

8. DevOps & CI/CD

GitHub Actions Workflow

Following the CI/CD tutorial, create `.github/workflows/build.yml`:

▶ Click to expand: GitHub Actions Workflow Example

CI/CD Requirements

Grade Level	CI/CD Requirements
Pass Level	- Basic GitHub Actions workflow that builds the project - Workflow triggered on push to main branch
Merit Level	- Automated testing in CI/CD - Workflow triggered on pull requests - Build and test status visible in GitHub
Distinction Level	- Code coverage reporting (Codecov or similar) - SonarCloud integration with quality gates - PostgreSQL service container for integration tests - (Optional) Doxygen documentation generation

SonarCloud Integration (Bonus)

Add SonarCloud analysis for code quality:

▶ Click to expand: SonarCloud Configuration

Source Control Best Practices

- Meaningful commit messages following convention
- `.gitignore` configured (no credentials, no bin/obj)
- Minimum 15 commits over project period
- Feature branches for major features (optional but encouraged)
- Pull requests with meaningful descriptions (optional)

9. AI Tool Usage & Documentation (Addresses LO4)

Encouraged AI Tools

> 💡 **Students are strongly encouraged to use AI tools to enhance productivity and learning:**

- **GitHub Copilot:** Code completion and suggestion
- **ChatGPT/Claude:** Problem-solving, debugging assistance, code explanation
- **Cursor/Windsurf:** AI-powered IDEs
- **Other AI coding assistants**

Requirements for Ethical AI Use

To demonstrate competent and ethical use of AI tools (LO4), students must:

1. **Be Transparent**
   - Document which AI tools were used
   - Identify which parts of the code were AI-assisted
   - Keep a log of significant AI interactions
2. **Show Control & Validation**
   - Demonstrate understanding of AI-generated code
   - Show how AI suggestions were evaluated and modified
   - Document cases where AI suggestions were rejected and why
3. **Critical Evaluation**
   - Don't blindly accept AI output
   - Test AI-generated code thoroughly
   - Verify that AI solutions follow best practices and design patterns

AI Usage Documentation (Required in Report)

Include a section in your report titled "AI Tool Usage" with:

▶ Click to expand: AI Documentation Format Example

> What NOT to Do:
>
> - ❌ **Copy-paste AI code without understanding it**
> - ❌ **Hide or omit AI usage**
> - ❌ **Accept AI suggestions that violate SOLID principles**
> - ❌ **Use AI-generated code without testing it**

Presentation Assessment of AI Use

During the presentation, tutors will:

- Ask you to explain AI-assisted code sections
- Probe your understanding of how the code works
- Ask why you chose to use/modify AI suggestions
- Evaluate whether you can defend design decisions

Grade Band	AI Usage Assessment
Excellent (90-100%)	Deep understanding of AI-generated code, thoughtful modifications, clear explanations of trade-offs
Good (70-89%)	Good understanding, some modifications, can explain most decisions
Satisfactory (50-69%)	Basic understanding, minimal modifications, can explain with prompting
Poor (40-49%)	Limited understanding, obvious blind acceptance of AI output
Fail (<40%)	Cannot explain AI-generated code, complete blind acceptance

10. Deliverables

1. Source Code (GitHub Repository)

Required Structure:

```
RentalApp/
├── .github/
│   └── workflows/
│       └── build.yml               # CI/CD pipeline
├── RentalApp/                      # MAUI app
├── RentalApp.Database/             # Database library
├── RentalApp.Migrations/           # EF Core migrations
├── RentalApp.Test/                 # xUnit tests
├── docker-compose.yml              # PostgreSQL container
├── .gitignore
├── README.md                       # Setup instructions
└── RentalApp.sln
```

README.md Contents:

- Project overview
- Setup instructions (Docker, database, dependencies)
- How to run the application
- How to run tests
- API endpoint documentation link
- Architecture overview

2. Documentation

Architecture Diagram:

- Component diagram showing MAUI App, Repositories, Services, Database
- Sequence diagram for rental workflow
- State diagram for rental states (if using State Pattern)

Code Documentation:

- XML documentation comments on public methods
- (Optional) Doxygen-generated HTML documentation

3. Test Artifacts

- xUnit test project with all test files
- Test coverage report (from CI/CD or local run)
- Evidence of DatabaseFixture usage
- Evidence of AAA pattern

4. CI/CD Pipeline

- GitHub Actions workflow file
- Successful workflow runs visible in GitHub
- Test results displayed in workflow
- (Optional) SonarCloud quality report

5. Written Report (Marked Quantitatively)

> ⚠ **Critical: The report is marked out of 100 and carries 20% weighting for the module. Marks are awarded for included items, not quality (quality is assessed in the presentation).**

Required Report Sections:

Section	Marks	Requirements
1. Cover Page	2	- Student name and ID - Project title - Submission date - GitHub repository URL
2. Architecture Documentation	20	- Component diagram (5 marks) - Database schema diagram (5 marks) - Sequence diagram for rental workflow (5 marks) - State diagram for rental states if implemented (5 marks bonus)
3. Feature Implementation Checklist	20	- List of all implemented features with checkboxes (10 marks) - Screenshots of each feature (10 marks)
4. Testing Documentation	15	- Test coverage report (screenshot/export) (5 marks) - List of test classes and test counts (5 marks) - Example test code (2-3 tests shown) (5 marks)
5. CI/CD Evidence	10	- GitHub Actions workflow file included (5 marks) - Screenshot of successful workflow run (5 marks)
6. Design Patterns Implementation	15	- MVVM pattern explanation with code example (5 marks) - Repository pattern explanation with code example (5 marks) - Service layer explanation with code example (5 marks) - (Bonus) State Pattern or MediatR explanation with code (5 marks bonus)
7. AI Tool Usage Documentation	15	- List of AI tools used (3 marks) - At least 3 significant AI interactions documented (9 marks - 3 marks each) - Reflection on AI usage (3 marks)
8. References	3	- Properly formatted references for external sources, libraries, and tutorials used

Total Report Marks: 100 marks (with bonus marks available for advanced features)

Report Format:

- PDF format
- Maximum 20 pages (excluding appendices)
- 12pt font, single or 1.5 line spacing
- Include page numbers
- Professional formatting

> 💡 **Note:** This is a **quantitative** marking scheme - you get marks for including the required items. The quality of your work (code quality, design decisions, understanding) is assessed in the **presentation**, not the report.

11. Assessment & Grading

This coursework has two components:

- **Written Report:** 100 marks, 20% of module grade (marked quantitatively)
- **Face-to-Face Presentation:** 100 marks, 50% of module grade (marked qualitatively)

Presentation Grading (100 marks, 50% module weighting)

> ⚠ **Critical: Marked qualitatively - focus on student's ability to EXPLAIN and DEFEND their work.**

Format:

- Face-to-face with tutor
- Recorded via Microsoft Teams
- 8-10 minutes: Student demonstrates and explains
- 8-10 minutes: Tutor questions
- Maximum 18 minutes total

Presentation Structure:

1. **Application Demonstration (3-4 minutes)**
   - Show the app running with all implemented features
   - Navigate through key workflows (item creation → search → rental → review)
   - Highlight any bonus features (State Pattern, MediatR, etc.)
2. **Architecture & Design Explanation (3-4 minutes)**
   - Explain your architectural decisions
   - Show and explain key design patterns (MVVM, Repository, Service Layer)
   - Walk through one complex code section in detail
   - Justify technology choices
3. **Testing & Quality Assurance (2 minutes)**
   - Show test coverage report
   - Explain testing strategy
   - Demonstrate CI/CD pipeline
   - Discuss quality metrics
4. **Questions from Tutor (8-10 minutes)**
   - Answer questions about design decisions
   - Explain code sections in detail
   - Defend architectural choices
   - Demonstrate understanding of AI-assisted code

Marking Criteria

Criteria	Marks	Excellent (90-100%)	Good (70-89%)	Satisfactory (50-69%)	Poor (40-49%)	Fail (<40%)
Understanding of Code (LO1)	25	Deep understanding of all code, can explain any section in detail	Good understanding, can explain most sections with minor prompts	Basic understanding, needs prompting for complex sections	Limited understanding, struggles to explain code	Cannot explain code obvious lack of understandi
Design & Architecture (LO1)	20	Excellent architectural decisions, clear justification for patterns used, sophisticated design	Good architecture, appropriate patterns, clear reasoning	Basic architecture, some patterns used appropriately	Poor architecture, patterns misused or absent	No clear architecture chaotic desi
Testing & Quality (LO3)	15	Comprehensive testing strategy, high coverage, quality metrics understood and applied	Good testing approach, reasonable coverage, understands metrics	Basic testing present, limited coverage, shallow understanding of metrics	Minimal testing, low coverage, doesn't understand quality assurance	No meaningful testing or quality measures
Development Process (LO2)	10	Excellent CI/CD implementation, clear development workflow, good Git practices	Good CI/CD, reasonable workflow, decent Git usage	Basic CI/CD, simple workflow, minimal Git usage	Poor process, broken CI/CD, bad Git practices	No process discipline, n working CI/
AI Tool Usage (LO4)	20	Transparent AI usage, thoughtful evaluation of suggestions, deep understanding of AI-generated code, clear modifications	Good AI transparency, evaluates suggestions, understands AI code, some modifications	Documents AI usage, basic evaluation, can explain AI code with prompting	Minimal AI documentation, limited understanding of AI output	No transparenc blind acceptance AI output, cannot expl AI code
Communication & Presentation	10	Clear, confident communication; well-structured presentation; answers questions thoroughly	Good communication; organised presentation; answers most questions well	Adequate communication; somewhat organised; answers questions with prompting	Poor communication; disorganised; struggles to answer questions	Incoherent presentatio cannot answer questions

Total Presentation Marks: 100 marks

> 💡 **Key Assessment Focus:**
>
> - **Not just "does it work?" but "can you explain WHY and HOW it works?"**
> - **Understanding over implementation** - A student with fewer features but deep understanding scores better than one with many features but shallow understanding
> - **AI Tool Ethics** - Transparent, controlled AI use scores highly; blind acceptance penalised
> - **Design Justification** - Can you defend why you chose specific patterns/approaches?

Grade Boundaries

70-100% (First Class): Excellent work demonstrating deep understanding, advanced features, sophisticated design
60-69% (2:1): Good work with solid understanding, Tier 2 features complete, appropriate patterns
50-59% (2:2): Satisfactory work with basic understanding, most Tier 2 features, some patterns
40-49% (3rd Class): Basic work meeting minimum requirements, Tier 1 complete, limited patterns
<40% (Fail): Incomplete work, missing core features, lack of understanding

12. Submission Guidelines

Submission Deadline

[To be specified - typically end of week 6]

What to Submit

1. **GitHub Repository URL**
   - Submit link to your public GitHub repository
   - Ensure repository is public (not private)
   - Repository must include all source code, tests, CI/CD workflow
2. **Written Report (PDF)**
   - **REQUIRED** - marked out of 100 (20% of module)
   - See Section 10 (Deliverables) for detailed report requirements
   - PDF format, maximum 20 pages
   - Submit via [submission system TBD]
3. **Presentation Booking**
   - Book a face-to-face presentation slot with your assigned tutor
   - Presentations will be held in [location TBD] during week [TBD]
   - Bring your laptop with application ready to demonstrate
   - Presentations are recorded via Microsoft Teams

Submission Checklist

Before submitting the report, verify:

- ☐ GitHub repository is public and accessible
- ☐ All required features implemented
- ☐ Tests run successfully (>60% coverage for Merit, >80% for Distinction)
- ☐ CI/CD pipeline passes (green checkmark in GitHub)
- ☐ README.md complete with setup instructions
- ☐ Report includes all required sections (see Section 10)
- ☐ Report is PDF format, under 20 pages
- ☐ No credentials or sensitive data in repository
- ☐ Code is well-commented
- ☐ GitHub shows minimum 15 commits over project period
- ☐ AI tool usage documented in report

Before the presentation, verify:

- ☐ Application runs smoothly on your laptop
- ☐ All features functional and can be demonstrated
- ☐ You can explain all code sections
- ☐ You understand AI-generated code and can defend it
- ☐ Test coverage report available to show
- ☐ CI/CD pipeline status visible in GitHub
- ☐ Architecture diagrams prepared (can be shown on screen)

Academic Integrity

- You may use StarterApp as your foundation (encouraged)
- You must cite external libraries (MediatR, NetTopologySuite for PostGIS)
- You may use AI tools (GitHub Copilot, ChatGPT) but must understand the code
- You may discuss concepts with peers but must write your own code
- Copying code from other students will result in academic misconduct proceedings
- Turnitin may be used for written documentation
- Code similarity checking will be performed

Late Submission Policy

[Specify university policy - typically penalised per day late]

Extensions

Students requiring extensions due to extenuating circumstances should contact Student Services and provide documentation.

13. Support Resources

Module Tutorials

All tutorials available in `/tutorials/csharp/`:

- **StarterApp:** Docker setup, PostgreSQL connection
- **MAUI Tutorial:** Building notes app, XAML basics
- **MVVM Pattern:** ViewModels, Commands, ObservableObject
- **Database Migrations:** Three-project structure, EF Core migrations
- **Unit Testing:** xUnit, AAA pattern, DatabaseFixture
- **CI/CD:** GitHub Actions, automated testing

External Resources

Official Documentation:

- .NET MAUI Documentation
- Entity Framework Core
- PostGIS Documentation
- xUnit Documentation
- GitHub Actions Documentation

Design Patterns:

- MediatR GitHub
- State Pattern Explanation
- Repository Pattern in C#

PostGIS with .NET:

- NetTopologySuite for EF Core
- Spatial Data in EF Core

Office Hours

[Specify instructor office hours and location]

Discussion Forum

[Specify if using Moodle forum, Teams channel, etc.]

Technical Support

- For Docker/environment issues: [IT support contact]
- For API issues: consult the API Reference document or test endpoints via the Swagger UI at `https://set09102-api.b-davison.workers.dev/`
- For GitHub Actions issues: Consult cicd tutorial or ask in forum

14. Timeline & Milestones

Duration: 4-6 weeks (half semester)

Week 1: Setup & Foundation

Learning Objectives:

- Set up development environment
- Understand StarterApp structure
- Implement basic item CRUD

Tasks:

- ☐ Download StarterApp and verify Docker setup
- ☐ Create your GitHub repository
- ☐ Implement three-project structure (App, Database, Migrations)
- ☐ Create database schema using EF Core migrations
- ☐ Implement Item model and basic views (list, detail)
- ☐ Implement ItemViewModel with Commands
- ☐ Create ItemRepository
- ☐ Integrate with API: GET /items, GET /items/{id}, POST /items

Deliverable: Basic app can list and create items

Week 2: Location & Search

Learning Objectives:

- Implement PostGIS spatial queries
- Abstract location services
- Create search functionality

Tasks:

- ☐ Implement ILocationService with device GPS integration
- ☐ Update Item model with GEOGRAPHY(POINT) location field
- ☐ Implement GetNearbyAsync in ItemRepository using PostGIS
- ☐ Create NearbyItemsPage and ViewModel
- ☐ Integrate with API: GET /items/nearby
- ☐ Add category filter to search
- ☐ Start writing unit tests (repositories, services)

Deliverable: Location-based search working

Week 3: Rental Workflow

Learning Objectives:

- Implement multi-state workflow
- Create service layer for business logic
- Handle complex state transitions

Tasks:

- ☐ Implement Rental model
- ☐ Create RentalService with business logic (date validation, double-booking prevention)
- ☐ Implement RentalRepository
- ☐ Create rental views: RentalsPage (incoming/outgoing)
- ☐ Integrate with API: POST /rentals, GET /rentals/incoming, GET /rentals/outgoing
- ☐ Implement approve/reject functionality (PATCH /rentals/{id}/status)
- ☐ Write comprehensive unit tests for RentalService
- ☐ Set up DatabaseFixture for integration tests

Deliverable: Full rental workflow operational

Week 4: Reviews & Testing

Learning Objectives:

- Implement feedback system
- Achieve comprehensive test coverage
- Set up CI/CD pipeline

Tasks:

- ☐ Implement Review model and repository
- ☐ Create ReviewsPage and ViewModel
- ☐ Integrate with API: POST /reviews, GET /items/{id}/reviews
- ☐ Complete unit test suite (ViewModels, Services, Repositories)
- ☐ Achieve >60% test coverage (Merit) or >80% (Distinction)
- ☐ Set up GitHub Actions workflow (build + test)
- ☐ Add PostgreSQL service container to CI/CD
- ☐ Verify pipeline passes

Deliverable: Reviews working, comprehensive tests, CI/CD pipeline green

Week 5: Advanced Features & Refactoring (Optional/Bonus)

Learning Objectives:

- Implement advanced design patterns
- Demonstrate Open/Closed Principle
- Refactor for clean code

Tasks:

- ☐ (Bonus) Implement State Pattern for rental states
- ☐ (Bonus) Implement MediatR for event-driven architecture
- ☐ Refactor existing code to eliminate smells
- ☐ Add code documentation (XML comments)
- ☐ Integrate SonarCloud (optional)
- ☐ Generate Doxygen documentation (optional)
- ☐ Polish UI/UX

Deliverable: Advanced patterns implemented, code refactored

Week 6: Documentation & Submission

Learning Objectives:

- Create professional documentation
- Finalise submission
- Prepare for presentation

Tasks:

- ☐ Write comprehensive README.md
- ☐ Create architecture diagrams (component, sequence, state)
- ☐ Write final report (all required sections)
- ☐ Perform final testing on all platforms
- ☐ Verify CI/CD pipeline passes
- ☐ Review submission checklist
- ☐ Submit repository link and report
- ☐ Book presentation slot
- ☐ Prepare presentation materials

Deliverable: Complete submission package, ready for presentation

---

> 💡 Final Reminders:
>
> - This is not just about building an app that works - it's about demonstrating professional software engineering practices
> - Quality is assessed through your presentation - be prepared to explain and defend your work
> - Use AI tools ethically and transparently - document your usage and demonstrate understanding
> - Ask questions early and often - use office hours and discussion forums
> - Start early to allow time for iterative development and testing

For API endpoint details, see the separate API Reference document.

Last modified: Monday, 9 March 2026, 8:27 PM